

arete-ui

A minimal, agent-driven Generative UI shell for enterprise apps — built on Google's A2UI v0.9 protocol.

Executive Summary

arete-ui is a React framework that lets end-users reshape their enterprise application UI through natural-language conversation with an agent, with every change gated by a visual diff before it commits. It is deliberately small: arete-ui ships an app **Shell**, a multi-surface **Page** workspace, a **Chat** panel, a **Visual Diff Engine**, and a **Page Operations Harness** that gives the agent typed, structural control of the workspace. Everything else — components, theming, the agent, auth, data, persistence — is plugged in from outside. The first reference plug-in is Freshworks' fw-dew design system, but arete-ui core has no dependency on it.

arete-ui builds on Google's **A2UI** (Agent-to-User Interface) v0.9 protocol and reuses A2UI's renderer and agent SDKs unchanged. Its net-new contribution is the shell pattern, the diff engine, and the structural-operation harness — none of which A2UI provides on its own.

The Problem

Enterprise apps (ERP, CRM, ITSM) are dense, rigid, and built for the median of many personas at once. The result is familiar:

- **One-size-fits-none layouts.** A page tuned for everyone is optimal for no one. Cognitive load rises, throughput drops.
- **High mutation cost.** Changing a layout, adding a panel, surfacing a new data stream all require frontend work, design review, and regression risk.
- **Administrative chokepoints.** Layout flexibility is gated behind admin roles. Power users get a handful of templates, never a workspace tuned to their actual day.

The result is a frozen UI that can't keep up with how individuals actually work.

The Solution

arete-ui shifts UI customization from an engineering ticket to a conversational act. A user says what they want — *"give me a panel of overdue approvals grouped by urgency"*, *"pin this widget to the Tickets page"*, *"make Reports a 3x3 grid"* — and an agent emits the change. Before anything mutates, the user sees a **visual diff**: green outlines on added components, red on removed, yellow on moved. One click commits; one click rejects.

[User intent] → [Agent] → [Shadow Surface + Diff Overlay] → [User approves] → [Live UI]

Three properties make this safe enough for enterprise use:

1. **Diff-gated by default.** The agent never silently mutates the user's workspace. Every structural and content change passes through the same approve/reject pipeline.

2. **Per-surface granularity.** Multiple pending diffs across multiple surfaces resolve independently — no all-or-nothing commits.
3. **Deterministic guardrails.** Lifecycle hooks let host apps reject, rewrite, or audit any agent-proposed change before it can paint, including RBAC and data-resolution policy.

Architectural Scope

arete-ui is intentionally narrow. It owns the **shell, the workspace, the chat, the diff, and the structural-op harness** — and nothing else.

In scope (core)	Out of scope (plugged from outside)
App shell with left tab rail, top-bar slot, dockable chat	Authentication, RBAC enforcement, business-data persistence
Multi-surface page workspace with configurable regions	The catalog of UI components (consumer plugs in any A2UI v0.9 catalog)
Visual Diff Engine (shadow surface + overlay highlights + approve/reject)	The agent runtime and LLM transport (consumer supplies the message stream)
Page Operations Harness (typed structural ops the agent can invoke)	Long-term persistence of layout / conversation / approval history
Lifecycle hooks (onBeforeApply, resolveDataPath, onUserAction, onPrompt, onPageOp)	Domain-specific components (ERP, CRM, ITSM widgets live in adapters, not core)

For every out-of-scope concern, arete-ui exposes a clean hook or pluggable interface; it never bundles policy.

Technical Foundation

Protocol target: A2UI v0.9

v0.9 is the current published, feature-complete A2UI specification with renderer support across React, Angular, Lit, and Flutter. v0.8 is closed legacy; v0.10 is in draft and v1.0 is roadmapped for Q4 2026. arete-ui v1 targets v0.9 and tracks v0.10 evolution without implementing it until v0.10 closes.

Dependencies and reuse (we do not rebuild A2UI)

arete-ui consumes A2UI's existing primitives as-is:

- **Renderer:** @a2ui/react/v0_9 (A2uiSurface, basicCatalog, createComponentImplementation) and @a2ui/web_core/v0_9 (MessageProcessor, SurfaceModel, SurfaceGroupModel, DataModel, ComponentsModel).
- **Agent SDK:** A2UI's Python and Kotlin agent SDKs already ship streaming parsing (A2uiStreamParser), inference strategies and prompt generation (A2uiSchemaManager, A2uiTemplateManager, InferenceStrategy), schema validation (A2uiValidator), payload repair (PayloadFixer), and a standard agent toolset (SendA2uiToClientToolset). Host apps reuse these directly.
- **Theming pattern:** A2UI's CSS-variable theming via :where(:root) is reused. arete-ui adds no new theming abstraction.

What arete-ui adds on top of A2UI

A2UI is a **content protocol**: surfaces, components, data, actions. It says nothing about:

- A multi-tab application shell with a dockable chat panel.
- A persistent multi-surface workspace (every A2UI sample today is a single-pane chat).
- Diffing or staging — `updateComponents` is applied immediately by the message processor.
- Structural operations on a workspace (pin a widget to a page, reshape a page's region layout).

These four gaps are exactly the surface arete-ui fills.

Differentiation from existing A2UI tools

Existing A2UI artifact	What it is	Why arete-ui is distinct
tools/composer/	CopilotKit-driven designer widget-authoring tool	arete-ui runs inside a live app for end-users , gates mutations through a diff, and defines an app shell
tools/editor/	One-shot single-prompt UI generator	arete-ui mutates a persistent multi-surface, multi-page workspace incrementally
samples/client/react/shell/	Single-pane chat → renders A2UI reply	arete-ui is shell-shaped (rail, tabs, dockable chat), not single-pane chat

Scope: What arete-ui Ships

arete-ui core is one React package: **four components** plus a **page-operations harness** plus a **lifecycle-hook API**.

1. <Shell> — application frame

The top-level container. Mounted once.

- **Left edge rail**: vertical column of tab icons. Consumer registers tabs as { id, icon, label, render }. Click switches the active tab. Icon-only, tooltip on hover, ~48–56px wide.
- **Top-bar slot**: fully consumer-controlled (`topBar=<MyTopBar />`). arete-ui ships no defaults for title, search, user menu — those belong to the host app.
- **Main content area**: renders the active tab's render output.
- **Right-edge chat dock**: optional. When `chatTab` is configured, the chat is full-page on its own tab AND simultaneously docked to the right edge on every other tab. The dock collapses to a vertical-strip rail with a chevron expand/collapse affordance. Dock state persists across tab switches.
- **State surface**: state + `onStateChange` lets consumers persist Shell state however they want (localStorage, backend, nothing). Core ships no persistence.

2. <Page> — multi-surface agent-driven workspace

A convenience component for tabs whose content is a multi-surface, agent-mutated workspace. **Optional** — any React component can live in a tab. <Page> is what consumers use when they want the agent → diff → approve loop on a multi-region layout.

- Wraps `SurfaceGroupModel` from `@a2ui/web_core/v0_9`.
- Consumer configures named docked regions; surfaces are routed to regions by `surfaceId→region` mapping.
- Each region renders an `A2uiSurface` from `@a2ui/react/v0_9`.
- Layout topology is serializable JSON (consumer persists it).
- Every surface inside <Page> is automatically wrapped in <DiffOverlay>.

3. <Chat> — chat panel (full-page or docked)

A single component instance powers both render modes: full-page on the chat tab, docked on other tabs.

- **Render modes:** "page" (full main area), "dock" (right edge, ~360–420px), "rail" (collapsed vertical strip).
- **Surface list:** each agent-emitted surface is appended chronologically. Each entry uses <A2uiSurface> with an optional user-prompt header. The list itself is plain React — not a new A2UI catalog widget; BasicCatalog already ships an in-surface List.
- **Chat input:** text + send button, plain React (not an A2UI catalog widget). On submit, calls a consumer-supplied onPrompt(text) callback. arete-ui does not own the agent transport.
- **Shared state:** conversation history is preserved across tab switches and dock/page/rail transitions.

4. <DiffOverlay> + Shadow Surface — Visual Diff Engine

The flagship feature. Used automatically by <Page>; exported as a primitive for consumers wrapping their own surfaces.

- **Shadow routing:** when an agent message (updateComponents / createSurface / deleteSurface) arrives for a surface wrapped by <DiffOverlay>, it is written to a **shadow SurfaceModel** instead of the live one.
- **Overlay highlights:**
 - green = added component
 - red = removed component (live still visible underneath, dimmed)
 - yellow = moved or attribute-changed component
- **Approve / Reject bar:** floating, anchor-configurable. Approve flushes shadow → live via the underlying MessageProcessor. Reject drops the shadow.
- **Tree-level diff:** computed using component IDs from A2UI's flat updateComponents list.
- **Per-surface granularity:** each surface gets its own pending shadow + approve/reject. Concurrent pending diffs resolve independently.
- **Diff hooks:** onProposed(diff), onApprove(diff), onReject(diff) — consumers wire to audit, undo, persistence.

5. Page Operations Harness — structural commands

A2UI's protocol mutates **surface contents** only. arete-ui adds typed structural commands the agent can invoke against the workspace itself.

Shipped operations (v1):

Operation	Purpose	
pinSurface(surfaceId, pageId, region?)	Promote a surface from the chat scroll into a page region	
unpinSurface(surfaceId)	Return a pinned surface to the chat scroll	
setPageLayout(pageId, layoutDescriptor)	Change a page's region layout (e.g. 3x3 grid, 2-column split, custom dock)	
moveSurface(surfaceId, targetRegion)	Move a pinned surface between regions on the same page	
`setPageRegion(pageId, regionId, surfaceId \`	Explicit slot assignment	

Design rules:

- These operations are **first-class arete-ui actions**, not A2UI messages. arete-ui defines their JSON schemas and ships their implementations.
- Consumers expose them to their agent however they want — typically as an ADK / OpenAI / vendor toolset, analogous to A2UI's SendA2uiToClientToolset. arete-ui ships only the schemas and implementations, not the agent-side toolset wrappers.
- **All structural ops flow through the Diff Engine by default** — "change layout to 3x3" previews the new layout before committing. Consumers can mark specific ops as autoApprove per-op or per-tab.
- All ops emit through the same onProposed / onApprove / onReject stream as content diffs, so audit and undo are unified across content and structure.

Lifecycle hooks

Exposed at the Shell level, threaded through every component and the harness:

Hook	Purpose
onBeforeApply(messages, ctx)	Sanitize, validate, RBAC-gate agent output before it touches any shadow. Return modified messages or null to reject.
resolveDataPath(path, ctx)	Wrap JSON Pointer resolution with enterprise auth, tenant prefixing, redaction.
onUserAction(action)	Audit hook on every A2UI action dispatch.
onPrompt(text)	Consumer routes the user's chat-input prompt to its own agent/transport.
onPageOp(op, ctx)	Fires for every page-harness operation; consumers can deny, transform, or log.

Pluggable surface (everything else lives outside)

- **Catalog:** any A2UI v0.9 Catalog instance. Reference plug-in (separate package, not in core): @arete-ui/adaptor-dew — maps @freshworks/dew-components to A2UI implementations via createComponentImplementation and ships @freshworks/dew-styles tokens as the theme. arete-ui core has zero fw-dew dependency.
- **Agent / transport:** consumer brings the A2UI message stream (SSE / WebSocket / A2A / mock). arete-ui exposes ingest(stream).
- **Page roster:** consumer declares which tabs exist, their icons/labels, and what renders in each.
- **Top-bar content:** consumer-owned React.
- **Persistence:** none in core. Layout JSON, conversation history, approval log, undo stack — all consumer-owned via the hook APIs.

Routing Model

How agent output finds its destination:

- **Default for agent replies with UI:** every reply lands as a new surface in the **chat scroll**. The agent cannot silently mutate an active page just because the user is looking at it.
- **Moving widgets onto a page is an explicit act:** the user says *"pin this widget to this page"* (or *"the second one"*, *"this card"*) and the agent invokes pinSurface(...) from the harness. The pin op flows through the Diff Engine — user approves, surface promotes from chat scroll into the page region.
- **Layout changes are agent-issued via the harness:** *"change this page into a 3x3 grid"* → setPageLayout(pageId, gridDescriptor) → preview overlay → approve commits.
- **Acknowledgement bubbles:** a structural op may optionally push a short text surface into the chat

scroll ("*Pinned KPI panel to top-left of Reports*"). arete-ui emits a default ack; consumers can override or suppress.

Reference Webapp: examples/erp-sandbox

A local-first React SPA that demonstrates the full plug-in surface end-to-end. **Not** part of arete-ui core.

- **Shell composition:** 1 chat tab + 2 workspace tabs (e.g., "Tickets", "Reports") using <Page>.
- **Catalog plug-in:** @arete-ui/adaptor-dew, consuming @freshworks/dew-components and @freshworks/dew-styles.
- **Mock agent:** emits canned updateComponents and page-op messages in response to a prompt fixture set:
 - "*Group approvals by urgency*"
 - "*Add an outstanding-invoices panel*"
 - "*Swap KPI card with sparkline*"
 - "*Pin the second card to Tickets*"
 - "*Make Reports a 3x3 grid*"
- **Persistence demo:** better-sqlite3 stores Shell state, conversation history, and approval log — wiring onStateChange and onApprove to a real backend. Used only by the demo, not by core.

The sandbox demonstrates: chat full-page on chat tab; chat docked on other tabs; chat minimized to rail; agent reply lands in chat scroll; pinSurface op overlays target region → approve → surface promotes; setPageLayout op previews 3x3 grid → approve.

Critical Files to Reference

- **Renderer primitives:** renderers/react/src/v0_9/, renderers/web_core/src/v0_9/processing/message-processor.ts, renderers/web_core/src/v0_9/state/ (SurfaceModel, SurfaceGroupModel, DataModel, ComponentsModel).
 - **Streaming + multi-surface reference:** samples/client/react/shell/.
 - **Custom-component registration pattern** (for @arete-ui/adaptor-dew, not core): samples/client/lit/custom-components-example/.
 - **Theming reference:** docs/guides/theming.md, renderers/react/src/v0_8/styles/ (CSS-var conventions).
 - **A2UI specification:** specification/v0_9/docs/a2ui_protocol.md, specification/v0_9/json/client_capabilities.json.
 - **Agent SDK reference:** agent_sdks/agent_sdk_guide.md, agent_sdks/python/src/a2ui/.
-

Out of Scope

- Authentication, RBAC enforcement, business-data persistence, raw API routing.
- ERP-specific components — those live in catalog adapters, never in core.
- Server-side LLM hosting or agent runtime.
- Layout editor UI (drag-to-rearrange panels) — agent-mediated only in v1.
- v0.10 / v1.0 A2UI support until v0.10 closes.
- Non-React framework support in v1; Angular / Lit / Flutter adapters are future work.

Delivery Plan & Verification Checklist

Workstream A: Core library (@arete-ui/core)

1. <Shell> chrome: left rail renders ≥ 3 tab icons; click switches active tab; top-bar slot accepts arbitrary consumer React.
2. <Chat> full-page mode: renders main area as vertical surface list + sticky input.
3. <Chat> dock mode: switching off chat tab moves the same chat instance to right edge; minimize collapses it to a vertical rail strip; conversation history preserved across switches.
4. <Page> topology: configure named docked regions on a non-chat tab; agent-created surfaces slot into regions per surfaceld mapping; configured regions render even when empty.
5. Shadow-diff loop: mock agent emits updateComponents → overlay renders with green / red / yellow outlines → Approve mutates live → Reject leaves live unchanged. Repeat for createSurface and deleteSurface.
6. Per-surface diff isolation: two surfaces with concurrent pending shadows resolve independently.
7. Lifecycle hooks: onBeforeApply rejecting a message stops shadow from painting; resolveDataPath prefix-injection resolves correctly through agent-bound paths; onApprove fires with the committed diff.
8. Page Operations Harness: pinSurface previews target region → approve promotes from chat to page. setPageLayout with a 3×3 grid descriptor previews → approve commits. Reject leaves state untouched in both cases.
9. Harness JSON schemas: every page op has a published JSON schema; wrapping into an ADK toolset (parallel to SendA2uiToClientToolset) is mechanical.
10. Conformance: any wrapped A2UI bits (MessageProcessor subclass etc.) pass relevant v0.9 conformance suites (agent_sdks/conformance/suites/).

Workstream B: Reference plug-ins and sandbox

11. @arete-ui/adaptor-dew separate package: maps @freshworks/dew-components to A2UI implementations via createComponentImplementation; ships @freshworks/dew-styles tokens as the bundled theme.
12. Catalog plug-in proof: swap BasicCatalog → @arete-ui/adaptor-dew without touching arete-ui core; same prompt fixtures still render and diff correctly.
13. examples/erp-sandbox: full Shell + Page + Chat + Diff loop with the prompt fixture set above. Mock agent + SQLite persistence demonstrate the hook wiring.
14. Agent reply routing: prompt typed in docked chat on a non-chat tab — reply lands as a new surface in the chat scroll, NOT silently on the page.